

CAMPUS NAVIGATION AND EVENT MANAGEMENT SYSTEM

ENSF 338

Term Project

Jacob Plourde, Benjamin Anderson, Asim Hamdan, Denis Sukhanov, Toan
Le, Alexander Gamble

April 15, 2026

TABLE OF CONTENTS

INTRODUCTION	1
CAMPUS MAP AND SHORTEST PATH NAVIGATION	1
Design Decisions	1
Campus Representation.....	1
Justification for This Representation.	1
Rejected Alternative.	1
Complexity Analysis.....	1
Insert Building.	1
Add Pathway.	2
Remove Pathway.....	2
Delete Building.	2
Find Shortest Path.....	2
Importing the Graph from a File.	3
Challenges and Lessons Learned	3
ROUTE HISTORY AND NAVIGATION UNDO	3
Design Decisions	3
Data Structure Choice	3
Justification for this Choice.....	3
Rejected Alternative	3
Complexity Analysis.....	3
Add History State (Push)	3
Undo Last Route (Pop).....	4
ROOM AND EVENT BOOKING SYSTEM	4
Design Decisions	4
Data Structure Choice	4
Justification for this Choice.....	4
Alternatives Considered: Unsorted Linked List	4
Complexity Analysis.....	5
Method 1: add_booking	5
Method 2: event_in_range	5

Method 3: remove_booking.....	5
Space Complexity:	5
Challenges and Lessons Learned	6
PRIORITY-BASED SERVICE QUEUE	6
Design Decisions	6
Complexity Analysis.....	6
Method 1: Enqueue	6
Method 2: Dequeue.....	6
Method 3: Peek (index 0).....	6
Space Complexity :	6
FAST BUILDING AND RESOURCE LOOKUP	6
Design Decisions	6
Data Structure Choice.	7
Justification for this Choice.	7
Rejected Alternative.	7
Complexity Analysis.....	7
Insertion / Deletion / Lookup. I	7
INCOMING REQUEST PROCESSING	7
Design Decisions	7
Data structure chosen	7
Why this choice is appropriate	7
Alternative data structure	7
Complexity Analysis.....	7
Method 1: enqueue	7
Method 2: dequeue	8
Challenges and Lessons Learned:	8
CONTRIBUTIONS	8
DEMONSTRATIONS	8

INTRODUCTION

The Campus Navigation and Event Management System is a cohesive software solution designed to handle spatial data, scheduling constraints, and rapid information lookup within a complex university environment. Developed in Python, the system models the physical campus as a network of buildings and weighted pathways to compute optimal routes. It integrates multiple specialized data structures to guarantee operational efficiency across its core features: an undirected weighted graph for shortest-path routing via Dijkstra's algorithm, a bounded Last-In-First-Out (LIFO) stack for session navigation undo capabilities, a self-balancing AVL tree for chronological event indexing, a priority queue for service request handling, and hash tables for instantaneous resource lookup. The primary objective of this project is to implement these functional requirements while critically analyzing and justifying the underlying algorithmic choices based on precise time and space complexity constraints.

CAMPUS MAP AND SHORTEST PATH NAVIGATION

Design Decisions

Campus Representation. The campus is represented internally by an undirected weighted graph that contains a dictionary, or hash table (which will be explained in a later section), to aggregate nodes that represent buildings. Edges between these nodes represent pathways between the buildings. The chosen structure for the edges is an adjacency list stored in each building node.

Justification for This Representation. A graph was chosen to represent the campus because graphs are best used in cases where each node (buildings) may have many neighbors, and a building may have many paths leading out of it to other buildings. Additionally, because shortest path navigation must be implemented, graph edge weights can represent the time or distance between buildings, and then Dijkstra's algorithm can be used to find the shortest path between any two buildings.

Rejected Alternative. One alternative to the current design was to store a hash table of edges in the campus class as opposed to using adjacency lists for each node. This would yield a greater space efficiency because each edge would only need to be stored once instead of twice (once for each node). It would also improve time complexity for determining if a path exists (and its weight, if so) between any two given nodes. However, because we need to find shortest paths, adjacency lists are superior due to the lower time complexity of finding all neighbors of a given node.

Complexity Analysis

Insert Building. Building insertions are hash table insertions. Their time complexities are as follows:

- Best case: $O(1)$, where there are no hashing collisions
- Average case: $O(1)$, because most hashing collisions are uncommon
- Worst case: $O(V)$, where the hashing collides with all present elements before finding a free spot

The space complexity is $O(1)$; no additional space is used.

Add Pathway. Adding pathways has the same best, worst, and average time complexity because it is a head insertion of a linked list, which is $O(1)$. The space complexity is $O(1)$ because no auxiliary space is used.

Remove Pathway. Time complexities are as follows:

- Best case: $O(1)$, where the list head is empty or we are removing a pseudograph edge and the method is early returned.
- Average case: $O(D)$ where D is the degree of the node, because it traverses edges until it finds the one to be removed.
- Average case: $O(D)$, where the edge to be removed is at the end of the linked list traversal or does not exist.

Space complexity is $O(1)$; no additional space is used.

Delete Building. Building deletions first iterate over all nodes, removing edges of the building to be deleted, then follow with a hash table deletion. The time complexities are as follows:

- Best case: $O(V)$, the campus graph contains no edges, leading to early returns, and there are no hashing collisions. All nodes must be iterated over, however.
- Average case: $O(V + E)$, because it is likely that every node, or almost every node has edges, which must be iterated over to find pathways between each building and the one being deleted.
- Worst case: $O(V^2)$, where it is a complete graph, and for each node that has its pathway removed, the node being deleted is at the end of its adjacency list.

The space complexity is $O(1)$; no additional space is used.

Find Shortest Path. Dijkstra's algorithm is used to find the shortest path between buildings. It also uses a priority queue to increase efficiency. The time complexities are as follows:

- Best case: $O(V \log(V))$, where there are no edges, but all nodes are still iterated over and enqueued / dequeued from the priority queue.
- Average case: $O((V + E) \log(V))$, where we iterate over unvisited nodes and dequeue from the priority queue ($V \log(V)$), and then potentially enqueue for each connected edge ($E \log(V)$)

- Worse case: $O(V^2 \log(V))$, same as average, but the graph is a complete graph, so there are V^2 edges.

Space complexity is $O(V + E)$, because we initialize the priority queue with all (V) nodes, dequeue once for each iteration, but enqueue potentially for every edge attached to that node.

Importing the Graph from a File. The best, worst, and average time complexities are same because the algorithm will always check and parse the file the same way; this complexity is $O(V + E)$ because the file format first lists nodes, which are parsed, and then lists edges, which are parsed. The space complexity is also $O(V + E)$ because the entire graph is created.

Challenges and Lessons Learned

One challenge I faced was deciding how to import graphs from a file. At first, the easiest solution seemed to be just using a binary data file, but that would not be as easily extendable or modifiable. I decided on creating my own simple graph file format that first lists buildings as “[id]__[name]__[latitude]__[longitude]” on newlines. Then there’s a separator line with “===” to indicate that following are the edge definitions. Edges are listed as “[id1]__[id2]__[weight]” on new lines. This can be parsed into a graph.

ROUTE HISTORY AND NAVIGATION UNDO

Design Decisions

Data Structure Choice: A bounded, array-based Stack (Last-In, First-Out).

Justification for this Choice: The undo operation inherently requires reverting to the most recently visited state before proceeding to older states. A LIFO stack architecture guarantees this exact chronological retrieval. To satisfy the project requirement of "at least 10 levels of undo" while strictly preventing unbounded memory growth during prolonged user sessions, the stack implementation is explicitly bounded to a maximum capacity (e.g., 50 states).

Rejected Alternative: A Queue (First-In, First-Out). This was rejected because it violates the chronological requirement of an undo system. A queue would revert the user's oldest navigation state first, rather than their most recent, breaking the intended user experience.

Complexity Analysis

Add History State (Push)

- **Best/Average Time Complexity:** $O(1)$ amortized.

- **Worst Time Complexity:** $O(k)$, where k is the bounded capacity (e.g., 50).
- **Space Complexity:** $O(1)$ per insertion, strictly bounded to a maximum of $O(k)$ total memory.
- **Justification:** Appending a new navigation state to the end of a Python list operates in $O(1)$ amortized time. If the stack exceeds the predefined capacity bound, the oldest element is removed using `pop(0)`. While `pop(0)` is structurally $O(k)$ as it shifts subsequent array elements, k is a fixed constant, effectively reducing the operational overhead to $O(1)$.

Undo Last Route (Pop)

- **Best/Average/Worst Time Complexity:** $O(1)$.
- **Space Complexity:** $O(1)$.
- **Justification:** Retrieving and removing the most recent state utilizes the `pop(0)` method on the end of the array. This requires no index shifting for the remaining elements, guaranteeing constant time execution regardless of the current stack size.

ROOM AND EVENT BOOKING SYSTEM

Design Decisions

Data Structure Choice

- Chosen data structure: A self-balancing AVL Tree (`Event_Index`), and Sorted Doubly LinkedList.

Justification for this Choice

- **Access Requirements:** Users frequently request events within some time interval ("Find out what is happening from 1:00 PM to 4:00 PM?"). A self-balancing AVL Tree allows to find the beginning of such an interval in $O(\log n)$ time.
- **Ordering Requirements:** Bookings should be ordered according to the time. An AVL Tree guarantees this ordering, and a sorted list provides constant-time access to the Next Upcoming Event – we can simply select the next event with respect to the **current_time** value.
- **Insertion Requirements:** The bookings for university rooms do not change every millisecond, so $O(\log n)$ operations required for self-balancing of an AVL Tree are negligible compared to the huge gain in search performance.

Alternatives Considered: Unsorted Linked List

- **Reason for rejection:** An unsorted linked list allows insertion in $O(1)$ but requires performing searches and interval queries in $O(n)$ time. Considering the number of bookings that can be made on a campus of a certain size, it will take quite long. Sorting allows reducing time complexity of a search to $O(\log n)$ using binary search, but maintaining order via built-in functions was rejected in favor of a **manual insertion algorithm**. This ensures the list remains always ordered $O(n)$ per insertion.

Complexity Analysis

Method 1: add_booking

- Time complexity is $O(1)$ for the best case, if the tree is empty. And the worst and case complexity is $O(\log n)$.
- **Justification:** To insert into an AVL tree, we traverse the height of the tree. Since the tree remains balanced, its height will be approximately $\log n$. The insertion sort technique has been employed here to maintain the order of the bookings. We simply traverse the sorted list to figure out the right location of the new booking so that it maintains the order. This requires $O(n)$ time in the worst case, but since inserting into the AVL tree only requires $O(\log n)$ time, it means that the cost of adding a booking is dominated by $O(n)$.

Method 2: event_in_range

- Average and worst-case time complexity is $O(\log n + k)$, where k is the number of events
- **Justification:** The idea here is to do a binary search in the AVL tree for the event that happens at or just after the `start_time`, which will require $O(\log n)$ time, and then traverse the tree to collect the k events starting from there to the `end_time` in an in-order traversal.

Method 3: remove_booking

- Worst-case time complexity is $O(n)$.
- **Explanation:** At present, the removal of a booking involves filtering the entire list (n) and then constructing an AVL tree again. While deleting from an AVL tree will cost us $\log n$ time, the current $O(n)$ approach guarantees that the tree remains perfectly balanced and simple to handle due to the infrequent nature of cancellations.

Space Complexity:

- Space complexity is $O(n)$.
- **Explanation:** We store each booking once in the list and once as a node in the AVL tree. The memory usage scales linearly with the number of bookings (n).

Challenges and Lessons Learned

- An important difficulty was writing code to rotate nodes in an AVL tree using a node body represented as a list (**node.body**). Converting abstractions like children from the left or right and height to list indexes required extensive debugging to prevent IndexError exceptions. However, it became clear that although Python lists are versatile, there are some situations where custom data structures, such as AVL trees, are essential. A further challenge was ensuring all operations were handled manually, and while we first did not do so in our add_booking function due to Python's sort() function, our team subsequently modified it.

PRIORITY-BASED SERVICE QUEUE

Design Decisions

- Binary Heap as a dynamic array to avoid $O(n)$ complexity on shifting elements
- Min-Heap so lowest int representation as priority level is processed first
- Recursive Implementation for heapify_up and heapify_down for readability
- Using tuples to store priority and item in order to compare priority to simplify the if statements

Complexity Analysis

Method 1: Enqueue

- Best Case is $O(1)$ when the lowest priority/largest int value is appended to the list and the heapify check fails immediately.
- Worst Case is $O(\log n)$ when the highest priority/smallest int value is appended to the list and the heapify check needs to swap from the leaf to the root.

Method 2: Dequeue

- Best Case and Worst Case are both $O(\log n)$ because the downward path from heapify down must be verified

Method 3: Peek (index 0)

- Best and worst case are $O(1)$ due to it being a simple indexing of the root node.

Space Complexity :

- $O(n)$ due to the nodes being stored in a single list

FAST BUILDING AND RESOURCE LOOKUP

Design Decisions

Data Structure Choice. A python dictionary (hash table) was chosen as the data structure for any data that requires fast insertion, deletion, or lookup.

Justification for this Choice. Dictionaries almost guarantee $O(1)$ insertion, deletion, and lookup times, which fit the requirements of this feature.

Rejected Alternative. No alternatives were considered for this feature because as said in the justification, hashing is the only way to achieve this requirement.

Complexity Analysis

Insertion / Deletion / Lookup. Insertions, deletion, and lookup operations for hash tables all have the same best, worst, and average case time complexity, because they are dependent only on the hashing algorithm. Their time complexities are as follows:

- Best case: $O(1)$, where there are no hashing collisions
- Average case: $O(1)$, because most hashing collisions are uncommon
- Worst case: $O(n)$, where the hashing collides with all present elements before finding a free spot (for insertion), or element (for deletion / lookup)

The space complexity is $O(1)$; no additional space is used.

INCOMING REQUEST PROCESSING

Design Decisions

Data structure chosen: Singly Linked list with head and tail pointers to create a queue

Why this choice is appropriate: By using this data structure there is a $O(1)$ enqueue and dequeue time complexity and uses first in first out.

Alternative data structure: An alternative data structure would be to use an array. The reason we didn't choose this option is because the worst case for both enqueue and dequeue is $O(n)$ because it requires either copying the array to increase size or shifting the array down.

Complexity Analysis

Method 1: enqueue

Best, average and worst-case time complexity is $O(1)$

Space complexity is $O(1)$

Justification: Time complexity is $O(1)$ since enqueueing is done using the tail pointer to append to the end of the queue.

Space complexity for this method is $O(1)$ since the only memory used is the new node that's created.

Method 2: dequeue

Best, average and worst case time complexity is $O(1)$

Space complexity is $O(1)$

Justification: Time complexity is $O(1)$ since dequeuing is done using the head pointer to remove from the beginning of the queue.

Space complexity for this method is $O(1)$ since the only memory used is pointer to head node and the next node.

Challenges and Lessons Learned:

A challenge this part is to properly account for the different cases that could occur when enqueueing and dequeuing from the queue.

CONTRIBUTIONS

Contributor	Features
Jacob Plourde	2.1 Campus Map and Shortest Path Navigation, and accompanying report entry and UI section 2.5 Fast Building and Resource Lookup, and accompanying report entry and UI section - Priority queue implementation
Benjamin Anderson	2.6 Incoming Request Processing, and accompanying report entry and UI section
Toan Le	2.2 Route History and Navigation Undo, accompanying report entry and UI section.
Asim Hamdan	Made the Majority of the classes.py file, which contained a mix of finished data structure classes and stub classes for the program features. This file was later split into multiple modules in the src/ directory.
Denis Sukhanov	2.3 Room and Event Booking System, developing an AVL Tree for efficient event indexing and range querying. Also integrated the backend logic into the user interface to support precise scheduling with hour and minute granularity.
Alexander Gamble	2.4 Priority-based Service Queue, and accompanying report entry and UI section - Complete Video

DEMONSTRATIONS

Demo Link: [URL Redirect](#)